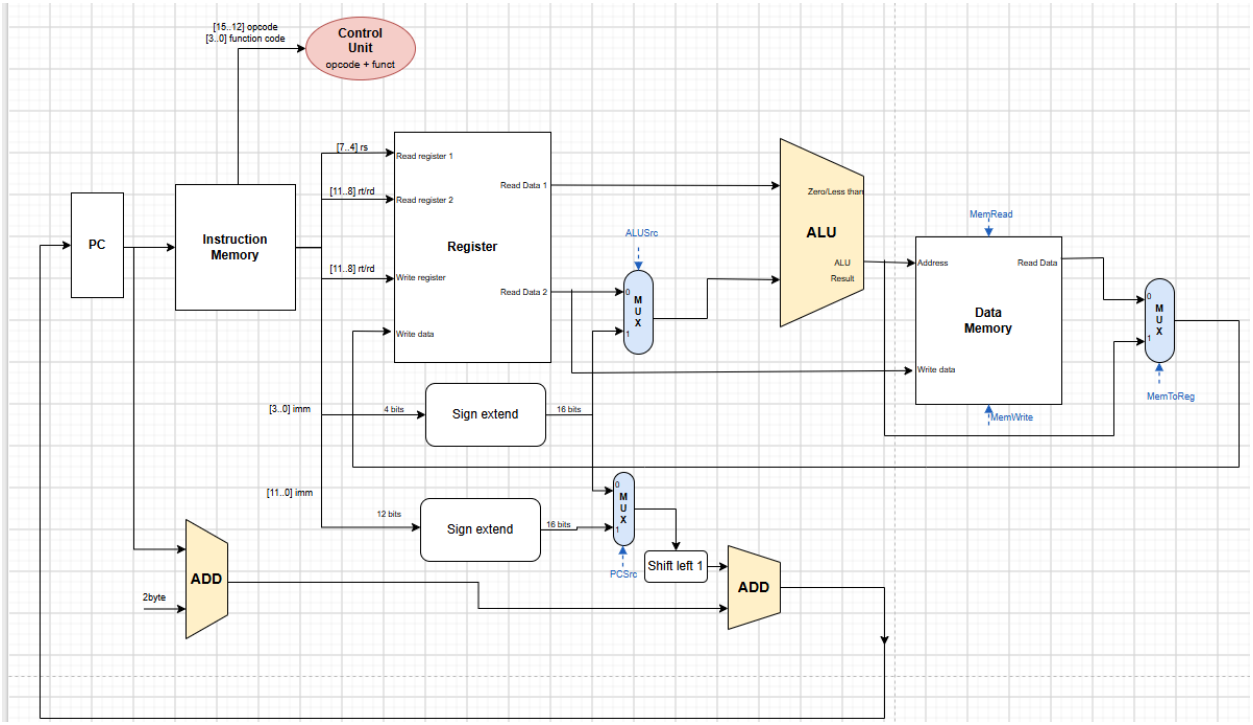
Design of different components

- Instruction Memory
  - Pulling the instruction from memory with the input as the PC address
  - Since we have 256 bytes of memory, each instruction takes 2 bytes (16 bits) → we have 256/2 = 128 slots of instructions
  - Each slot has an index from 0 → 127 (7 bits)
- Data Memory
  - With the input as the result from the ALU, memread and memwrite from the Control Unit
    - If memread = 1 → read from address (address == ALU result), and store it to register
    - If memwrite = 1 → write the value (from register) to the memory address (address == ALU result)
- Register File
  - Getting the bits of each register from the instruction memory
    - Bit [7:4]: rs
  - According to the bit input to get the value of the register
- ALU
  - Zero flag - returns 1 when 2 registers are equal
  - R type - when opcode == 0000 → do the task according to function code
  - I type - add immediate or branching
  - J type - jump to an address
- Wiring
  - Place to wrangle all the wires together and instantiate all the separate modules to coordinate the run. I believe if you wrote all the modules in one big code file and didn't instantiate each one separately you would run into a memory issue on the FPGA board.
  - The most important aspects of the wiring file are the inputs and outputs these are pivotal for testing. First, a clk is needed to synchronize the flow of data. Second, a reset flag this is important to start our program at PC zero. Lastly, the only output pc_out this is what all our testing is dependent on we know if we had a successful run if this ends with 0x72 and a failed run if it's greater than 0x74.
  -
- Sign Extension
  - Sign extension 4bit - input is always 4bits and output is the sign extension to 16bits. In a signed number the top bit, in this case the 4th bit, is the determining factor in whether or not it's positive or negative. If the fourth bit is 1 it's negative so the top 12 bits need to be 'extended' with ones. While a 0 bit needs to be filled with 12 zeroes.
  - Sign extension 12bit - only difference is the top 4 bits of this input need to be extended not 12.

    - Note on extension just calculate what in hex you need to add the input value to get the desired result.
- Control Unit
    - All flags for each type of instruction broken down by R type I type and J type
    - R type instructions only need a signal for writing the a register or if it's immidiate instruction then it also needs Alu_src so the ALU understands that it's doing and imm calculation
    - I type in this ISA are load word and store word, separate them by opcode sw only needs to write to memory (men_write) and the ALU operation of computing the memory offset. While lw needs to store its result in a register (reg_write) and read from the memory.
        - Both branches can be handled together. I decided to split them for ease of understanding, beq is taken when zero flag form the ALU is 1 and bne is taken when zero flag is 0.
    - J type flag for the PC so it knows to take the 12 bit sign extended value.
- Multiplexer(s)
    - Quick note the wiring file makes multiple different mux's by calling them a different name so only one component of code is necessary.
    - Call_wb_mux - switches between the normal data path of using register to now select the load word data path using data memory
    - Call_alu_src_mux - handles the sign-extended immediate values that are inputs to the ALU decides whether or not to use an imm value (for addi instruction).
    - Call_offset_mux - chooses between a 12-bit sign-extended value or a 4-bit sign-extended value, dealing with jump and branch instructions respectively.
- Program Counter
    - The pc should be able to take in a 16 bit target that it will use to add the pc's current address to this new 'target'. The pc has two options: either move the pc address a default amount this ISA has it 2bytes or it moves the pc address a given amount depending on jump and branch. The pc doesn't do any sign extension or calculation to figure out what instruction told it to move it a certain am not will just compute the address.

System architecture design



System integration

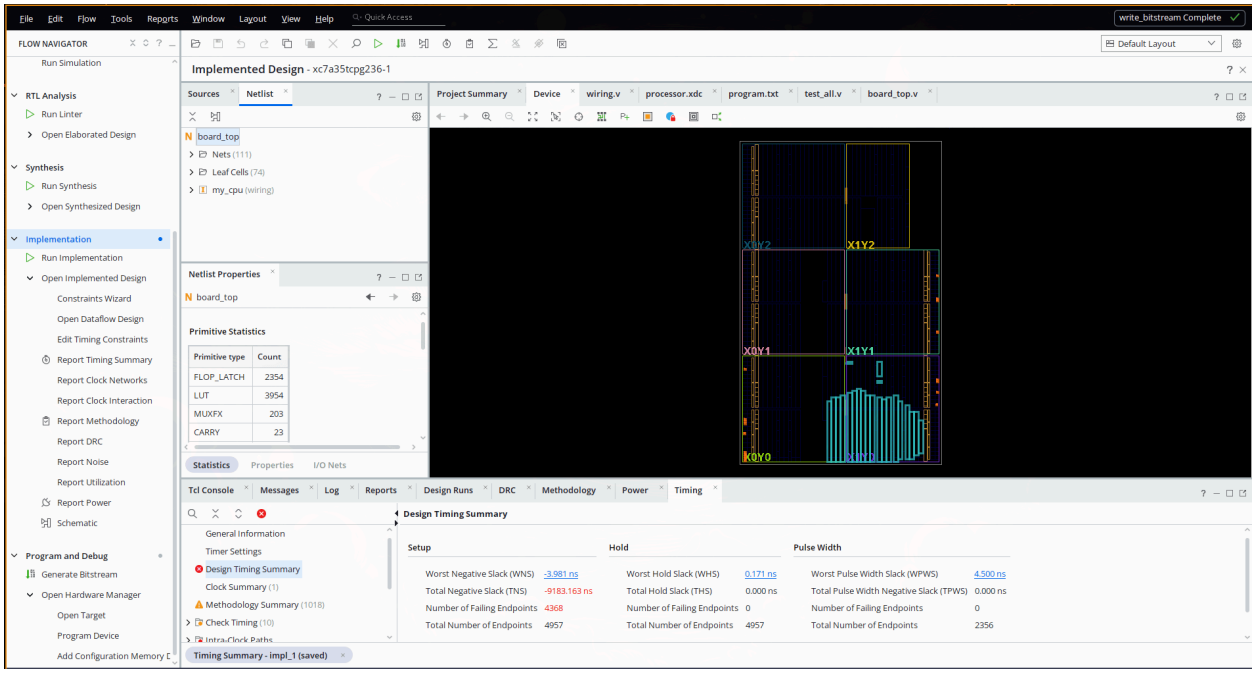
- Each component of the 16 bit processor look at its inputs waiting for it to see a change in value as soon as one is updated, the component will complete its operation using that data. For example, the control unit waits for the instruction memory to decode the next instruction. Once it sends the opcode bits to the control, that component can make a determination on which control signal to flip.

System testing

- Testbenches, one test for each module
    - General structure of each testbench follows: set the input values for that component read for those values to settle then in a if else control block display pass or fail on a comparison you hand computed as being correct.
    - More challenging tests are components that require a clock in the testbench itself such as: the data memory, pc, register file, wiring. For each of these a clock period needs to me set and before each test in the bench a clock edge has to be selected it's important to use '#1;' to make sure that the data has completed its path through that component.

- Unified testbench
    - The instruction memory allows for the input of instructions through a .txt file if said file contains the hex code of each instruction you wish to run.
    - A unified testbench only needs one test_all.v code file. Its job is to monitor the pc and set a clock to define the amount of time each test is given to run.
    - The program.txt file holds all the tests necessary for each instruction. For example to test the add instruction you need two values to add together and a correct value to compare against to see if your processor computed correctly. The program.txt file also gives you a proper fail condition which will inform you of which instruction failed. Each comparison has a specific address; it jumps the pc too in the event the processor calculates the wrong result.

Include snapshots of hardware



Simulation results - (Not exhaustive more testbenches in sourcecode)

- PC testbench

```
# run 1000ns
PASS: pc = 0000 after reset
PASS: pc = 2
PASS: pc = 4
PASS: pc = 6
PASS: pc = 40
PASS: pc = 42
PASS: pc = 44
PASS: pc = 21
PASS: pc = 23
pc testbench done
$finish called at time : 86 ns : File "/home/noetherverse/Projects/cse490_16bit_processor/vivado/16bit_processor.srcs/sim_1/new/pc-test.v" Line 65
INFO: [USF-XSim-96] XSim completed. Design snapshot 'pc_test_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:15 ; elapsed = 00:00:09 . Memory (MB): peak = 12882.629 ; gain = 85.859 ; free physical = 5242 ; free virtual = 179152
```

- Data Memory testbench

```
# run 1000ns
PASS: data = aabb
PASS: data = bb00
PASS: data = 4321
PASS: data = aabb
INFO: [USF-XSim-96] XSim completed. Design snapshot 'data_memory_test_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:11 ; elapsed = 00:00:09 . Memory (MB): peak = 12912.652 ; gain = 29.988 ; free physical = 4682 ; free virtual = 178639
```

- Sign Extension testbench

```
# run 1000ns
PASS: extend = 0002
PASS: extend = ffff
PASS: extend = fff8
PASS: extend = 0000
sign_extension done
INFO: [USF-XSim-96] XSim completed. Design snapshot 'sign_extension_test_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:09 ; elapsed = 00:00:08 . Memory (MB): peak = 12912.652 ; gain = 0.000 ; free physical = 4733 ; free virtual = 178743
```

- Unified testbench file test_all - this is the same tests that run on the FPGA board

```
launch_simulation: Time (s): cpu = 00:00:09 ; elapsed = 00:00:09 . Memory (MB): peak = 12912.652 ; gain = 0.000 ; free physical = 4348 ; free virtual = 178343
run all
PASS: all 10 instructions verifie
All Test done
$finish called at time : 2006 ns : File "/home/noetherverse/Projects/cse490_16bit_processor/vivado/16bit_processor.srcs/sim_1/new/test_all.v" Line 36
```

- Further explanation need to press the Run All button in the top bar to see the displayed output other approach is looking at the final value of the pc_out

